MiniTactic

Design specification for a small tactic-based theorem prover

Discrete mathematics - propositional logic - first-order logic - typed sets - induction

Prepared as an implementation-oriented design document

Core invariant
Tactics are untrusted. The kernel is trusted. Every accepted theorem has a kernel-checked proof object, and that proof object records whether it used constructive or classical principles.

Document map

- Sections 1-4 define the product goal, architecture, and foundation.
- Sections 5-12 define the language, proof objects, natural-deduction rules, and tactics.
- Sections 13-18 cover first-order logic, typed sets, induction, simplification, grammar, and diagnostics.
- Sections 19-23 give implementation milestones, internal data structures, unification, the standard library, and a Codex-ready implementation prompt.

1. Product goal

MiniTactic is a small tactic-based theorem prover designed for students. It should compile and run locally, and the same core should also compile to WebAssembly so it can be served in a browser-based teaching environment.

The intended curriculum coverage is the core material of a discrete mathematics course: propositional logic, first-order logic, equality, typed set theory, relations and functions, natural numbers, and basic structural induction principles.

The distinctive teaching feature is a mode switch between constructive and classical reasoning. The system should allow students to attempt the same theorem in each mode and see exactly which proof rule crosses the constructive/classical boundary.

- The prover should be small enough that students and instructors can understand its behavior.
- The kernel should remain intentionally minimal and trusted.
- The tactic layer should improve usability but should never be trusted without kernel checking.
- The language should avoid advanced features such as full dependent types, type classes, universe polymorphism, and high-powered automation.

2. High-level architecture

Use a standard proof-assistant architecture with a small proof-checking kernel and an untrusted elaboration/tactic layer.

```
source file
   ↓
lexer / parser
   ↓
surface AST
   ↓
elaborator
   ↓
core formulas, terms, proof scripts
```

```
   ↓
tactic engine
   ↓
proof object with no holes
   ↓
kernel checker
   ↓
accepted theorem
```

The kernel should know only typed terms, formulas, contexts, primitive proof rules, accepted axioms/theorems, and the current logic mode. It should not know about user-friendly tactics except insofar as tactics generate proof objects using kernel constructors.

The web/wasm version should call the same core library as the native CLI. The core crate should not depend on filesystem, terminal I/O, browser APIs, networking, or OS-specific assumptions.

```
mini-tactic/
  crates/
    mini_core/        # AST, parser, elaborator, kernel, tactics
    mini_cli/         # command-line checker / REPL
    mini_wasm/        # wasm-bindgen wrapper
  web/
    src/              # editor UI, goal pane, diagnostics
  examples/
    prop.mt
    fol.mt
    sets.mt
    nat_induction.mt
  tests/
    kernel/
    tactics/
    integration/
```

Component	Responsibility	Trusted?
Parser	Convert source text to surface AST with spans.	No
Elaborator	Resolve names, infer simple arguments, desugar syntax.	No
Tactic engine	Transform goals and fill proof holes.	No
Kernel	Check final proof object against statement and mode.	Yes
CLI	Run checker, display diagnostics, support local use.	No
Wasm wrapper	Expose check/goals/step API to browser UI.	No

3. Logic foundation

The recommended foundation is typed first-order intuitionistic logic with an explicit classical extension. This is expressive enough for a discrete mathematics course while avoiding the complexity of full dependent type theory or untyped ZF set theory.

Use typed sets rather than an untyped set-theoretic universe. For example, Set Nat, Set Person, Set (Pair Nat Nat), and List Nat should all be distinct and type checked.

3.1 Syntactic categories

```
Type
```

```
Term
Formula
```

3.2 Type representation

```
enum Type {
    PropSchema,              // only for theorem schemas, not object-level quantification
    Base(Symbol),            // Person, Color, Vertex, etc.
    Nat,
    Bool,
    Pair(Box<Type>, Box<Type>),
    List(Box<Type>),
    Set(Box<Type>),
}
```

Important distinction: Prop in theorem parameters is a schema-level proposition variable. It is not an object-level type over which students quantify. This keeps the object language first-order while still allowing theorem schemas such as P /\ Q -> Q /\ P.

```
theorem and_comm (P Q : Prop) : P /\ Q -> Q /\ P := by
  intro h
  split
  exact h.right
  exact h.left
```

4. Term language

Terms are ordinary first-order terms plus typed set expressions. Pair, list, and set-builder features can be delayed if the first milestone is propositional logic only.

```
enum Term {
    Var(Name),
    Const(Name),

    App(Box<Term>, Vec<Term>),

    Pair(Box<Term>, Box<Term>),
    Fst(Box<Term>),
    Snd(Box<Term>),

    Zero,
    Succ(Box<Term>),

    Nil(Type),
    Cons(Box<Term>, Box<Term>),

    EmptySet(Type),
    Singleton(Box<Term>),
    Union(Box<Term>, Box<Term>),
    Inter(Box<Term>, Box<Term>),
    Diff(Box<Term>, Box<Term>),

    SetBuilder {
        var: Name,
        var_type: Type,
        body: Formula,
    },
}
```

Example terms and types:

```
empty : Set Nat
{0} : Set Nat
A ∪ B : Set Nat
A ∩ B : Set Nat
A \ B : Set Nat
{ x : Nat | x in A /\ x in B } : Set Nat
```

5. Formula language

The formula language should include standard intuitionistic connectives, first-order quantifiers, equality, and typed set membership.

```
enum Formula {
    True,
    False,

    AtomPred(Name, Vec<Term>),   // P(x), R(x,y), Even(n)
    Eq(Term, Term),
    In(Term, Term),              // x in A

    Not(Box<Formula>),
    And(Box<Formula>, Box<Formula>),
    Or(Box<Formula>, Box<Formula>),
    Implies(Box<Formula>, Box<Formula>),
    Iff(Box<Formula>, Box<Formula>),

    Forall {
        var: Name,
        var_type: Type,
        body: Box<Formula>,
    },

    Exists {
        var: Name,
        var_type: Type,
        body: Box<Formula>,
    },
}
```

Internally, some connectives can be treated as definitional sugar, but keeping them as separate AST nodes initially improves diagnostics and pretty-printing.

```
not P      := P -> False
P <-> Q   := (P -> Q) /\ (Q -> P)
A subset B   := forall x, x in A -> x in B
```

6. Surface syntax

A small source file should be readable by students and should resemble the proof-state style they see in class.

```
mode constructive

sort Person

const alice : Person
const bob   : Person

pred Likes(Person, Person)

theorem self_imp (P : Prop) : P -> P := by
  intro h
  exact h

theorem and_comm (P Q : Prop) : P /\ Q -> Q /\ P := by
  intro h
  split
  exact h.right
  exact h.left
```

6.1 Commands

```
mode constructive
```

```
mode classical

sort Name

const c : T

func f : A -> B
func g : A -> B -> C

pred P(T1, ..., Tn)

def name ... := ...

axiom name : Formula

theorem name : Formula := by
  tactics
```

Theorem schemas may have proposition, predicate, type, and term parameters. Type parameters are schema-level parameters, not object-language terms.

```
theorem name (P Q : Prop) (A : Type) (x : A) : Formula := by
  tactics
```

7. Constructive/classical mode switch

The kernel should be constructive by default. Classical mode should not silently change the meaning of every rule; instead, it should permit a small set of named classical proof principles.

```
classical_axiom em :
  P \/ not P

classical_axiom dne :
  not not P -> P

classical_axiom by_contra :
  (not P -> False) -> P
```

7.1 Mode tracking

```
enum LogicMode {
    Constructive,
    Classical,
}

struct Theorem {
    name: Name,
    params: Vec<Param>,
    statement: Formula,
    proof: Proof,
    mode_used: LogicMode,
}
```

Mode combination rule:

```
constructive + constructive = constructive
constructive + classical    = classical
classical    + anything     = classical
```

- A theorem proved constructively can be used in classical mode.
- A theorem proved classically cannot be used in constructive mode unless the surrounding file or proof explicitly permits classical reasoning.
- Every theorem should report the strongest mode used by its proof.

7.2 Teaching examples

Constructively valid:

```
mode constructive

theorem not_not_em (P : Prop) : not not (P \/ not P) := by
  intro h
  apply h
  right
  intro p
  apply h
  left
  exact p
```

Classically valid:

```
mode classical

theorem em_example (P : Prop) : P \/ not P := by
  by_cases h : P
  left
  exact h
  right
  exact h
```

The same proof should fail in constructive mode with a diagnostic that says by_cases uses excluded middle for P and requires classical mode.

8. Kernel proof objects

The kernel should check explicit natural-deduction proof objects. The tactic engine constructs these objects; the kernel validates them.

```
enum Proof {
    Hyp(Name),

    TrueIntro,

    FalseElim {
        proof_false: Box<Proof>,
        target: Formula,
    },

    AndIntro(Box<Proof>, Box<Proof>),
    AndElimLeft(Box<Proof>),
    AndElimRight(Box<Proof>),

    OrIntroLeft {
        proof_left: Box<Proof>,
        right_formula: Formula,
    },
    OrIntroRight {
        left_formula: Formula,
        proof_right: Box<Proof>,
    },
    OrElim {
        proof_or: Box<Proof>,
        left_name: Name,
        left_case: Box<Proof>,
        right_name: Name,
        right_case: Box<Proof>,
        target: Formula,
    },

    ImpIntro {
        hyp_name: Name,
        hyp_formula: Formula,
        body: Box<Proof>,
    },
```

```
    ImpElim {
        proof_imp: Box<Proof>,
        proof_arg: Box<Proof>,
    },

    ForallIntro {
        var: Name,
        var_type: Type,
        body: Box<Proof>,
    },
    ForallElim {
        proof_forall: Box<Proof>,
        arg: Term,
    },

    ExistsIntro {
        witness: Term,
        proof_body: Box<Proof>,
        exists_formula: Formula,
    },
    ExistsElim {
        proof_exists: Box<Proof>,
        witness_name: Name,
        hyp_name: Name,
        body: Box<Proof>,
        target: Formula,
    },

    EqRefl(Term),
    EqSubst {
        eq_proof: Box<Proof>,
        proof_body: Box<Proof>,
        target: Formula,
    },

    TheoremRef {
        name: Name,
        type_args: Vec<Type>,
        term_args: Vec<Term>,
        formula_args: Vec<Formula>,
    },

    NatInd {
        var: Term,
        motive: Formula,
        base_case: Box<Proof>,
        step_var: Name,
        ih_name: Name,
        step_case: Box<Proof>,
    },

    Classical {
        rule: ClassicalRule,
        args: Vec<Proof>,
        target: Formula,
    },
}
```

8.1 Kernel checker signature

```
fn check_proof(
    env: &Env,
    ctx: &Context,
    proof: &Proof,
    expected: &Formula,
    allowed_mode: LogicMode,
) -> Result<LogicMode, KernelError>
```

The return value is the strongest mode actually used by the proof: Constructive or Classical. This lets the kernel reject a theorem declared or used constructively if the proof object contains a classical rule.

9. Natural-deduction rules

The first kernel should implement the standard introduction and elimination rules below. These rules are enough for propositional logic, first-order logic, equality, and the initial set-theory layer.

9.1 Implication

```
Gamma, h : A |- p : B
----------------------
Gamma |- lambdah. p : A -> B

Gamma |- f : A -> B
Gamma |- a : A
----------------------
Gamma |- f a : B
```

9.2 Conjunction

```
Gamma |- p : A
Gamma |- q : B
----------------------
Gamma |- (p, q) : A /\ B

Gamma |- p : A /\ B
----------------------
Gamma |- p.left : A

Gamma |- p : A /\ B
----------------------
Gamma |- p.right : B
```

9.3 Disjunction

```
Gamma |- p : A
----------------------
Gamma |- left p : A \/ B

Gamma |- q : B
----------------------
Gamma |- right q : A \/ B

Gamma |- p : A \/ B
Gamma, h1 : A |- q1 : C
Gamma, h2 : B |- q2 : C
----------------------
Gamma |- cases p of h1 => q1 | h2 => q2 : C
```

9.4 Falsehood

```
Gamma |- p : False
----------------------
Gamma |- absurd p : C
```

9.5 Quantifiers

```
Gamma, x : T |- p : P(x)
----------------------
Gamma |- fun x => p : forall x : T, P(x)

Gamma |- p : forall x : T, P(x)
Gamma |- t : T
----------------------
Gamma |- p t : P(t)

Gamma |- t : T
```

```
Gamma |- p : P(t)
----------------------
Gamma |- exists_intro t p : exists x : T, P(x)

Gamma |- p : exists x : T, P(x)
Gamma, x : T, h : P(x) |- q : C
x not free in C
----------------------
Gamma |- exists_elim p (x,h => q) : C
```

9.6 Equality

```
Gamma |- refl(t) : t = t

Gamma |- e : a = b
Gamma |- p : P(a)
----------------------
Gamma |- subst e p : P(b)
```

10. Tactic engine

The tactic engine should manipulate proof holes. Each tactic fills the current hole with a proof constructor containing zero or more new holes. At the end of a proof script, there must be no holes, and the completed proof object is checked by the kernel.

```
struct ProofState {
    root: ProofWithHoles,
    goals: IndexMap<GoalId, Goal>,
}

struct Goal {
    id: GoalId,
    context: Context,
    target: Formula,
    mode: LogicMode,
}

enum ProofWithHoles {
    Hole(GoalId),
    Done(Proof),
    Node(...),
}
```

Example: if the current goal is Gamma |- A -> B, then the tactic intro h fills the hole with an ImpIntro proof node and creates a new subgoal Gamma, h : A |- B.

```
ImpIntro {
  hyp_name: h,
  hyp_formula: A,
  body: Hole(new_goal)
}
```

11. Core tactics

11.1 Minimal tactic set

Tactic	Purpose
intro h	Introduce an implication hypothesis or a universally quantified variable.
exact e	Solve the goal with an exact proof expression.

Tactic	Purpose
assumption	Solve from a matching context hypothesis.
apply e	Use an implication or universally quantified theorem backward.
split	Prove a conjunction or iff by subgoals.
left	Choose the left side of a disjunction.
right	Choose the right side of a disjunction.
cases h	Eliminate conjunction, disjunction, existential, or falsehood.
exists t	Provide a witness for an existential goal.
refl	Prove an equality by reflexivity.
rewrite h	Rewrite using equality.
unfold name	Unfold a transparent definition.
simp	Unfold simple definitions and reduce obvious propositions.
exfalso	Change the current target to False.
contradiction	Close from False or from P and not P.
induction n	Apply natural-number induction.

11.2 Classical-only tactics

Tactic	Classical principle
by_cases h : P	Excluded middle for P.
by_contra h	Proof by contradiction / double-negation elimination.
classical	Locally allow classical reasoning.
push_neg	Mode-sensitive negation simplification.

The tactic contradiction should not be classical by itself. From P and not P, deriving False and then any target is constructively valid.

12. Propositional examples

```
mode constructive

theorem imp_trans (P Q R : Prop) : (P -> Q) -> (Q -> R) -> P -> R := by
  intro hpq
  intro hqr
  intro hp
  apply hqr
  apply hpq
  exact hp


theorem and_comm (P Q : Prop) : P /\ Q -> Q /\ P := by
  intro h
  split
  exact h.right
  exact h.left
```

```
theorem or_comm (P Q : Prop) : P \/ Q -> Q \/ P := by
  intro h
  cases h with
  | left hp =>
      right
      exact hp
  | right hq =>
      left
      exact hq
```

12.1 Constructive versus classical examples

Constructively valid:

```
mode constructive

theorem not_not_em (P : Prop) : not not (P \/ not P) := by
  intro h
  apply h
  right
  intro p
  apply h
  left
  exact p
```

Classically valid:

```
mode classical

theorem em (P : Prop) : P \/ not P := by
  by_cases h : P
  left
  exact h
  right
  exact h


theorem dne (P : Prop) : not not P -> P := by
  intro hnn
  by_contra hn
  apply hnn
  exact hn
```

13. First-order logic

First-order logic adds typed variables, predicate symbols, universal quantification, and existential quantification. Predicate variables should be permitted at the schema level for reusable theorems.

```
theorem forall_and_left
  (A : Type)
  (P Q : A -> Prop)
  : (forall x : A, P(x) /\ Q(x)) -> forall x : A, P(x) := by
  intro h
  intro x
  exact (h x).left


theorem exists_and_left
  (A : Type)
  (P Q : A -> Prop)
  : (exists x : A, P(x) /\ Q(x)) -> exists x : A, P(x) := by
  intro h
  cases h with
  | intro x hx =>
      exists x
      exact hx.left


theorem not_exists_to_forall_not
```

```
  (A : Type)
  (P : A -> Prop)
  : not (exists x : A, P(x)) -> forall x : A, not P(x) := by
  intro h
  intro x
  intro hp
  apply h
  exists x
  exact hp
```

A useful classical-only example is not (forall x : A, P(x)) -> exists x : A, not P(x). This should fail in constructive mode and succeed in classical mode.

14. Typed set theory layer

The set-theory layer should be typed and extensional. Set operations are defined by membership expansion.

```
A : Set T
x : T
x in A : Prop
A subset B : Prop
A ∪ B : Set T
A ∩ B : Set T
A \ B : Set T


x in empty           <-> False
x in {a}         <-> x = a
x in A ∪ B       <-> x in A \/ x in B
x in A ∩ B       <-> x in A /\ x in B
x in A \ B       <-> x in A /\ not (x in B)
A subset B           <-> forall x, x in A -> x in B
```

Set extensionality can be a standard axiom or foundational theorem of the typed set layer.

```
axiom set_ext
  (T : Type)
  (A B : Set T)
  : (forall x : T, x in A <-> x in B) -> A = B
```

14.1 Set examples

```
mode constructive

theorem inter_subset_left
  (T : Type)
  (A B : Set T)
  : A ∩ B subset A := by
  intro x
  intro hx
  exact hx.left


theorem subset_refl
  (T : Type)
  (A : Set T)
  : A subset A := by
  intro x
  intro hx
  exact hx


theorem union_subset
  (T : Type)
  (A B C : Set T)
  : A subset C -> B subset C -> A ∪ B subset C := by
  intro hAC
  intro hBC
  intro x
  intro hx
```

```
  cases hx with
  | left hxA =>
      apply hAC
      exact hxA
  | right hxB =>
      apply hBC
      exact hxB


theorem inter_comm
  (T : Type)
  (A B : Set T)
  : A ∩ B = B ∩ A := by
  apply set_ext
  intro x
  split
  intro hx
  split
  exact hx.right
  exact hx.left
  intro hx
  split
  exact hx.right
  exact hx.left
```

15. Natural numbers and induction

Start with built-in natural numbers, zero, successor, and a kernel-level induction principle exposed through an induction tactic.

```
type Nat
const 0 : Nat
func succ : Nat -> Nat


nat_ind :
  P(0) ->
  (forall k : Nat, P(k) -> P(succ(k))) ->
  forall n : Nat, P(n)
```

Expose induction as a structured tactic:

```
induction n with
| zero =>
    ...
| succ k ih =>
    ...
```

For first implementation, make Nat.add a built-in with rewrite rules rather than supporting arbitrary recursive definitions.

```
def add : Nat -> Nat -> Nat
rules
  add(0, m)       => m
  add(succ(n), m) => succ(add(n, m))


theorem add_zero_right (n : Nat) : n + 0 = n := by
  induction n with
  | zero =>
      refl
  | succ k ih =>
      simp
      rewrite ih
      refl
```

15.1 Structural induction later

```
type List T
```

```
const [] : List T
func cons : T -> List T -> List T

list_ind :
  P([]) ->
  (forall x : T, forall xs : List T, P(xs) -> P(cons(x, xs))) ->
  forall xs : List T, P(xs)


theorem append_nil_right
  (T : Type)
  (xs : List T)
  : xs ++ [] = xs := by
  induction xs with
  | nil =>
      refl
  | cons x xs ih =>
      simp
      rewrite ih
      refl
```

16. Definitions and simplification

Support transparent logical abbreviations and computation rules separately.

16.1 Logical abbreviations

```
def subset (A B : Set T) : Prop :=
  forall x : T, x in A -> x in B
```

16.2 Computation rules

```
def add : Nat -> Nat -> Nat
rules
  add(0, m)       => m
  add(succ(n), m) => succ(add(n, m))
```

The simp tactic should remain predictable in the first version. It should perform only obvious definitional expansions and built-in computation steps.

```
x in empty            ■ False
x in A ∪ B         ■ x in A \/ x in B
x in A ∩ B         ■ x in A /\ x in B
x in A \ B         ■ x in A /\ not (x in B)
A subset B            ■ forall x, x in A -> x in B
add(0, m)          ■ m
add(succ(n), m)    ■ succ(add(n, m))
```

17. Parser grammar sketch

```
File        ::= Command*

Command     ::= "mode" Mode
              | "sort" Ident
              | "const" Ident ":" Type
              | "func" Ident ":" FunctionType
              | "pred" Ident "(" TypeList ")"
              | "def" Ident ParamList? ":" TypeOrFormula ":=" Expr
              | "axiom" Ident ParamList? ":" Formula
              | "theorem" Ident ParamList? ":" Formula ":=" ProofBlock

Mode        ::= "constructive" | "classical"

ProofBlock  ::= "by" TacticBlock

TacticBlock ::= Tactic*
```

```
Tactic      ::= "intro" Ident
              | "exact" ProofExpr
              | "assumption"
              | "apply" ProofExpr
              | "split"
              | "left"
              | "right"
              | "cases" ProofExpr CaseBlock?
              | "exists" Term
              | "refl"
              | "rewrite" ProofExpr
              | "unfold" Ident
              | "simp"
              | "exfalso"
              | "contradiction"
              | "by_cases" Ident ":" Formula
              | "by_contra" Ident
              | "induction" Ident InductionBlock?

Formula     ::= Formula "->" Formula
              | Formula "<->" Formula
              | Formula "\/" Formula
              | Formula "/\" Formula
              | "not " Formula
              | "forall" Ident ":" Type "," Formula
              | "exists" Ident ":" Type "," Formula
              | Term "=" Term
              | Term "in" Term
              | Term "subset" Term
              | PredApp
              | "(" Formula ")"
              | "True"
              | "False"
```

Students should not be required to type Unicode. Provide ASCII alternatives.

Unicode	ASCII alternative
/\	/\
\/	\/
->	->
<->	<->
forall	forall
exists	exists
in	in
subset	subset
not	not

18. Diagnostics and proof-state display

Diagnostics are central to the educational value of the system. The system should explain why a tactic failed, not merely say that it failed.

18.1 Application mismatch

```
error: cannot apply `h`

current goal:
  Q
```

```
`h` has type:
  P -> R

To use `h`, the conclusion must match the current goal.
Expected conclusion:
  Q
Actual conclusion:
  R
```

18.2 Classical rule in constructive mode

```
error: classical tactic used in constructive proof

The tactic:
  by_contra h

uses the classical principle:
  ((P -> False) -> False) -> P

Current theorem is declared constructive.
Possible fixes:
  - change the theorem to `mode classical`
  - prove `not not P` instead of `P`
  - add a decidability assumption for `P`
```

18.3 Existential witness type mismatch

```
error: witness has wrong type

goal:
  exists x : Nat, P(x)

provided witness:
  alice : Person

expected:
  Nat
```

18.4 Goal pane

```
1 goal

T : Type
A B C : Set T
hAC : A subset C
hBC : B subset C
x : T
hx : x in A ∪ B
|- x in C
```

After simp at hx, the pane could display:

```
1 goal

T : Type
A B C : Set T
hAC : forall x : T, x in A -> x in C
hBC : forall x : T, x in B -> x in C
x : T
hx : x in A \/ x in B
|- x in C
```

19. Implementation milestones

19.1 Milestone 1: propositional constructive kernel

- Parser for theorem declarations.
- Prop schema variables.
- Formulas: True, False, And, Or, Implies, Not.
- Proof objects and kernel checker.
- Tactics: intro, exact, assumption, apply, split, left, right, cases, exfalso, contradiction.

Acceptance tests:

```
P -> P
P /\ Q -> Q /\ P
P \/ Q -> Q \/ P
(P -> Q) -> (Q -> R) -> P -> R
not (P \/ Q) -> not P /\ not Q
not not (P \/ not P)
```

This should fail constructively:

```
P \/ not P
```

19.2 Milestone 2: classical mode

- Theorem mode tracking.
- by_cases.
- by_contra.
- Classical axiom usage reporting.
- Constructive/classical import restrictions.

19.3 Milestone 3: first-order logic

- Types and term variables.
- Predicate symbols.
- Universal and existential quantifiers.
- intro for forall, exists for exists, cases for exists, and apply with simple instantiation.

19.4 Milestone 4: equality and rewriting

- Term equality.
- refl.
- Substitution.
- rewrite.
- Simple congruence for function applications.

19.5 Milestone 5: typed sets

- Set T.
- Membership, subset, union, intersection, difference, empty set, singleton.
- Set extensionality.

19.6 Milestone 6: natural numbers and induction

- Nat, 0, and succ.
- Primitive recursive addition.
- simp for computation rules.
- induction tactic.

19.7 Milestone 7: Wasm and web UI

Expose a small API from the core crate and wrap it for WebAssembly.

```
pub fn check_file(source: &str) -> CheckResult;
pub fn goals_at(source: &str, position: Position) -> GoalResult;
pub fn run_tactic(source: &str, theorem: Name, tactic_index: usize) -> StepResult;


#[wasm_bindgen]
pub fn check(source: &str) -> JsValue {
    let result = mini_core::check_file(source);
    serde_wasm_bindgen::to_value(&result).unwrap()
}
```

- Source editor.
- Diagnostics panel.
- Current goals panel.
- Theorem list.
- Mode indicator.
- Proof dependency and classical-use report.

20. Internal data structures

20.1 Environment

```
struct Env {
    sorts: HashMap<Name, SortDecl>,
    consts: HashMap<Name, ConstDecl>,
    funcs: HashMap<Name, FuncDecl>,
    preds: HashMap<Name, PredDecl>,
    defs: HashMap<Name, DefDecl>,
    theorems: HashMap<Name, Theorem>,
    axioms: HashMap<Name, Axiom>,
}
```

20.2 Context

```
struct Context {
    term_vars: Vec<TermBinding>,
    proof_vars: Vec<ProofBinding>,
}

struct TermBinding {
    name: Name,
    ty: Type,
}

struct ProofBinding {
    name: Name,
    formula: Formula,
}
```

20.3 Diagnostics

```
struct Span {
    start: usize,
    end: usize,
}

struct Diagnostic {
    span: Span,
    severity: Severity,
    message: String,
    notes: Vec<String>,
}
```

20.4 Binding representation

Implement binding carefully. Recommended options are de Bruijn indices internally or a locally nameless representation. A named-only implementation is easier initially but tends to produce subtle capture-avoiding substitution bugs.

21. Unification strategy

The first version needs only modest unification. For apply, support formula-variable matching, term metavariable matching, predicate-variable matching for schemas, and no higher-order unification.

Example:

```
h : forall x : A, P(x) -> Q(x)
goal: Q(a)

apply h
```

The system should instantiate x := a and create subgoal P(a).

21.1 Apply algorithm

Instantiate schema parameters with fresh metavariables.

Peel off leading forall binders.

Peel off implications A1 -> A2 -> ... -> C.

Unify C with the current goal.

Emit subgoals A1, A2, ...

Build the proof object using ForallElim and ImpElim.

When inference fails, require explicit arguments, for example apply h at a or exact h(a)(hp).

22. Standard library outline

```
Std.Prop
  and_comm
  and_assoc
  or_comm
  imp_trans
  not_not_em
  de_morgan_constructive_1
  de_morgan_classical_1

Std.FOL
  forall_and_left
  forall_and_right
  exists_and_left
```

```
  not_exists_to_forall_not
  classical_not_forall_to_exists_not

Std.Eq
  eq_symm
  eq_trans
  congr_arg

Std.Set
  subset_refl
  subset_trans
  inter_subset_left
  inter_subset_right
  subset_union_left
  subset_union_right
  union_comm
  inter_comm
  set_ext

Std.Nat
  add_zero_right
  add_zero_left
  add_succ_right
```

Every standard-library theorem should be annotated as constructive or classical. The UI should report classical dependencies clearly.

```
Theorem uses classical reasoning:
  - excluded middle
```

23. Codex-ready implementation prompt

The following prompt is suitable as a starting instruction for an implementation assistant.

```
Implement a small tactic-based theorem prover in Rust.

The system should have a core crate named mini_core. It should implement a typed first-order
 intuitionistic logic with a classical extension.

Architecture:
- Parser -> surface AST -> elaborator -> tactic engine -> proof object -> kernel checker.
- The kernel is trusted. Tactics are untrusted and must produce proof objects that the
kernel checks.
- The core crate must not depend on filesystem, network, terminal, OS threads, or browser
APIs.
- Later, the same core crate will be used by a CLI and a wasm-bindgen wrapper.

Initial milestone:
Implement only propositional logic.

Formula AST:
- True
- False
- Atom(Name)
- Not(Box<Formula>)
- And(Box<Formula>, Box<Formula>)
- Or(Box<Formula>, Box<Formula>)
- Implies(Box<Formula>, Box<Formula>)

Proof AST:
- Hyp(Name)
- TrueIntro
- FalseElim(proof_false, target)
- AndIntro(left, right)
- AndElimLeft(proof)
- AndElimRight(proof)
- OrIntroLeft(proof_left, right_formula)
- OrIntroRight(left_formula, proof_right)
- OrElim(proof_or, left_name, left_case, right_name, right_case, target)
- ImpIntro(hyp_name, hyp_formula, body)
- ImpElim(proof_imp, proof_arg)
```

```
- TheoremRef(name, formula_args)
- Classical(rule, args, target)

Logic modes:
- Constructive
- Classical

The kernel checker should have this signature:

fn check_proof(
    env: &Env,
    ctx: &Context,
    proof: &Proof,
    expected: &Formula,
    allowed_mode: LogicMode,
) -> Result<LogicMode, KernelError>

It should return the strongest mode used by the proof.

Implement these tactics using proof holes:
- intro
- exact
- assumption
- apply
- split
- left
- right
- cases
- exfalso
- contradiction
- by_cases, classical-only
- by_contra, classical-only

A theorem declared constructive must be rejected if its final checked proof uses Classical
mode.

Implement parser support for examples like:

mode constructive

theorem and_comm (P Q : Prop) : P /\ Q -> Q /\ P := by
  intro h
  split
  exact h.right
  exact h.left

theorem not_not_em (P : Prop) : not not (P \/ not P) := by
  intro h
  apply h
  right
  intro p
  apply h
  left
  exact p

mode classical

theorem em (P : Prop) : P \/ not P := by
  by_cases h : P
  left
  exact h
  right
  exact h

Use spans for diagnostics. Add integration tests:
- and_comm succeeds constructively.
- imp_trans succeeds constructively.
- not_not_em succeeds constructively.
- em fails constructively.
- em succeeds classically.
- by_contra fails constructively.
- by_contra succeeds classically.

After propositional logic works, extend to first-order terms, quantifiers, equality, typed
sets, and natural number induction.
```

24. Example full files

24.1 Constructive file

```
mode constructive

theorem and_comm (P Q : Prop) : P /\ Q -> Q /\ P := by
  intro h
  split
  exact h.right
  exact h.left

theorem imp_trans (P Q R : Prop) : (P -> Q) -> (Q -> R) -> P -> R := by
  intro hpq
  intro hqr
  intro hp
  apply hqr
  apply hpq
  exact hp

theorem not_not_em (P : Prop) : not not (P \/ not P) := by
  intro h
  apply h
  right
  intro p
  apply h
  left
  exact p

sort Person

pred Student(Person)
pred Happy(Person)

theorem forall_and_left
  (P : Person -> Prop)
  (Q : Person -> Prop)
  : (forall x : Person, P(x) /\ Q(x)) -> forall x : Person, P(x) := by
  intro h
  intro x
  exact (h x).left

theorem exists_and_left
  (P : Person -> Prop)
  (Q : Person -> Prop)
  : (exists x : Person, P(x) /\ Q(x)) -> exists x : Person, P(x) := by
  intro h
  cases h with
  | intro x hx =>
      exists x
      exact hx.left
```

24.2 Classical file

```
mode classical

theorem em (P : Prop) : P \/ not P := by
  by_cases h : P
  left
  exact h
  right
  exact h

theorem dne (P : Prop) : not not P -> P := by
  intro hnn
  by_contra hn
  apply hnn
  exact hn
```

24.3 Set file

```
mode constructive

theorem inter_subset_left
  (T : Type)
  (A B : Set T)
  : A ∩ B subset A := by
  intro x
  intro hx
  exact hx.left

theorem union_subset
  (T : Type)
  (A B C : Set T)
  : A subset C -> B subset C -> A ∪ B subset C := by
  intro hAC
  intro hBC
  intro x
  intro hx
  cases hx with
  | left hxA =>
      apply hAC
      exact hxA
  | right hxB =>
      apply hBC
      exact hxB
```

25. Design recommendations

Build the system in layers. The early versions should prove a small number of useful theorems reliably rather than chasing automation or expressiveness.

Propositional constructive prover.

Classical mode switch.

First-order quantifiers.

Equality and rewriting.

Typed sets.

Natural numbers and induction.

Lists and structural induction.

Wasm/web UI.

```
Most important invariant
Every accepted theorem has a kernel-checked proof object, and that proof object records
whether it used classical principles. This invariant supports the central teaching goal:
students can compare constructive and classical proofs and see exactly where a proof crosses
 the boundary.
```

26. Reference links

These references are useful background for implementation style and deployment. They are not required to understand the design.

- Lean reference manual: kernel and proof-checking architecture - https://lean-lang.org/doc/reference/latest/
- Coq/Rocq documentation: proof terms, tactics, and the de Bruijn criterion - https://rocq-prover.org/
- Rust wasm32-unknown-unknown target documentation - https://doc.rust-lang.org/rustc/platform-support/wasm32-unknown-unknown.html

- wasm-bindgen documentation - https://rustwasm.github.io/docs/wasm-bindgen/